

Data-driven RSI: Eval、Synthetic Data Ladder 与轨迹产线

RSI 描述一种能够持续递归的自我改进：当前模型参与创造更强的后继模型，后继模型再推进下一轮。本文从其中最容易开始实验的一段——Data——往下推。

Eval

Synthetic data

Harness-Evolve

RSI 想做到什么

RSI 是 Recursive Self-Improvement，递归自我改进。当前模型参与创造更强的后继模型；后继模型继承这套能力，继续发现问题、提出改进并训练下一代。每一轮都改进模型，也改进产生模型的方法。循环可以持续下去，才有“递归”的含义。

今天，这套循环主要由基础模型团队维持。OpenAI、Moonshot AI (Kimi)、智谱 AI (GLM) 等团队的研究者不断寻找能力边界、搭建 eval、生产数据、运行训练，再决定下一版往哪里走。模型负责完成一次次任务，人负责改进制造模型的过程。

RSI 想把后一部分也逐步交给模型。即使没有人持续守在循环里，LLM 也能找到值得解决的能力缺口，提出可以验证的改进，生产下一轮训练经验，并判断后继模型是否真的更强。后继模型还要能够重复这套过程。

这要求系统具备几种相连的能力：找到有价值的问题；把问题转成训练干预；用独立证据确认效果；根据结果决定下一轮。只会生成更多样本，循环走不远。只会在固定 benchmark 上涨分，方向也会很快耗尽。RSI 的核心，是整个改进过程逐轮积累。

为什么先从 Data 开始

Data 提供了一个更容易落地的起点。下一次关键的算法变化会出现在哪里，一个想法放大以后还灵不灵，都很难提前判断。数据实验则很具体：让模型先做一批没见过的任务，找到稳定的失败，再把这些失败变成新的任务和轨迹，训练回下一版模型。

关键在于让模型的失败改变下一批数据，新增 token 只是手段。如果这个 loop 能持续运转，模型就已经开始参与决定自己下一轮学什么。它离完整意义上的 RSI 还有距离，但可能是最先能够被认真验证的一部分。

沿着这条思路往下走，会遇到三个问题：

- “数学更强”“coding 更强”这种模糊目标，怎么评估到足以指导下一步？
- 一批 synthetic data 看起来不错，怎么知道目标模型能不能学到，又该做多少？
- 过去靠人摸索出来的合成经验，怎么变成一条可以反复跑的轨迹产线？

这篇就只谈这三件事。

先给几个工作定义

本文只研究这条长循环中的一段：当前模型参与发现能力缺口、生产下一轮训练经验；训练得到的后继模型在未参与生产的新任务上取得可复现的提升。模型暂时不需要在一次推理中直接重写参数。

开放目标评估（open-ended evaluation）允许目标保持宽泛，例如数学研究能力。每轮留下的证据仍要具体：任务、轨迹、失败原因、验证结果和下一步的数据处方都要能复查。

本文把这类数据生产方式暂称为 **Harness-Evolve**。Harness 定义模型工作的环境；Evolve 在这个环境里产生、修改和筛选多条候选轨迹；最后再通过 SFT 把选出的过程训练进目标模型。权重更新发生在 SFT 阶段。

为什么一定会走到 synthetic data

只要把 Data-driven 往前推一步，synthetic data 就躲不开。

这在 post-training 里已经很明显了。很多 SFT 样本、偏好对、批评与重写、verifier 筛过的 rollout、tool-use trajectory，都由模型、规则、工具和人共同构造。现在不少能力提升，靠的就是这些被设计过的经验。

PT 也开始有类似的味道。自然数据仍是地基。随着清洗、去重和筛选不断改进，下一份增量数据会更多来自重写、转换、难度控制、推理补全、代码执行筛选，或者把已有材料重新变成一个值得学的任务。PT 和 post-training 的数据边界会越来越模糊。

自然数据仍是地基，给了模型知识、语言和对世界的基本认识。它很少会照着某个 checkpoint 的失败生长。模型刚好卡在哪一步，互联网通常不会立刻送来一万条难度合适、反馈清楚的练习。

这里所说的数据墙，指的是高质量增量数据的增长开始跟不上模型和算力的胃口。这些数据需要对下一版模型有用、没有被反复训练过，质量也要过关。对公开文本存量与训练需求的估算也指出，高质量人类文本可能成为继续扩大训练规模的约束之一（Villalobos et al., 2024）。撞墙日期很难精确预测，但新增 token 已经不能稳定换来新增能力。

Will DePue 把互联网称为 deep learning 的“一次性补贴”（DePue, 2026）。过去几十年，人类写下网页、代码、论文和讨论时并没有考虑模型训练，这些材料后来恰好组成了一份巨大、便宜而且跨领域的数据集。下一阶段很难再复制同样的偶然性。互联网之外更有价值的部分，主要是组织内部的工作流、专家的隐性判断、没有被记录的失败，以及只能在真实环境里观察到的过程。

因此，数据问题至少有两个不同的轴：**volume** 是已有领域里还需要多少高质量样本；**coverage** 是哪些任务、工具、边界情况和长程过程根本没有被记录。Synthetic data 很适合补 volume，却不能凭空恢复不存在的知识。要补 coverage，仍然需要人和真实环境提供锚点，再让模型围绕这些锚点展开、组合和探索。

稀缺的是下一步刚好有用的经验。

如果模型还要靠 scaling 往前走，就要参与制造下一轮经验：把自己的失败变成任务，把工具和 verifier 的反馈变成过程，再把其中有用的轨迹训练回去。Synthetic data 因此会成为 RSI 的重要前置。

Synthetic data 也可能更快地复制旧东西。一个模型批量写出它本来就会写的答案，数量再大也未必有用；让它一边生产、一边自评，还可能把自己的偏好越放越大。Eval、Ladder 和轨迹产线需要一起设计。

1. Eval 要持续生长

固定 benchmark 的分辨率通常不会永久保持。当前沿模型接近题目上限、题型被反复适配、样本可能回流进训练时，它对最强系统的区分力会下降；不过，不同 benchmark 的饱和速度和原因并不相同。对 60 个常用文本 benchmark 的研究中，近一半已经表现出较高或很高的饱和度（Akhtar et al., 2026）。Contamination 又让分数更难解释，而且影响会随模型与 benchmark 改变（Singh et al., 2024）。

过去的解法是继续由人造更难的 benchmark，但这件事越来越贵。Humanity’s Last Exam 从七万多道候选题里筛出 2,500 道，动员了近千名专家并做了多轮审核（Center for AI Safety & Scale AI, 2025）。由此得到一个仍需验证的判断：如果模型开始以更短的周期迭代，人类定义任务、写题、维护答案和更新验证集的速度，迟早会跟不上模型自己变强的速度。

Eval 要从固定题库变成持续迭代的系统。目标可以是模糊的，比如“数学研究能力更强”或“能完成更长的 coding task”；模型则从上一轮 failure map 出发，寻找新的能力边界，提出任务，制造反例，调节难度，持续长出新的 benchmark 和 validation set。AutoBench 已经把 benchmark creation 写成了一个由模型搜索“重要、新颖、困难”数据集的过程（Li et al., 2025）。

模型参与搭 benchmark，同时也要保留独立的验证边界。它可以提出候选题、测试和 verifier；用于训练和找错的 active benchmark（工作集）可以不断变化，用于确认进步的 private validation（私有验证集）则应与训练产线隔离，并对被评 checkpoint 隐藏。独立生成器、环境证据或人工审核负责检查题目是否有效、答案是否正确、数据是否泄漏。在生成私有验证前冻结被评 checkpoint 很有用，还需要配合这些独立检查。

模型可以自己出下一套题，进步则由独立证据确认。

Self-evaluation 让模型维护一条不断移动的能力边界。Self-validation 让模型参与构造检验，再由它无法提前迎合的证据完成确认。



一轮 eval 最后应该产出一份能驱动下一步的 **failure record**。Score 可以保留，failure record 还要写清任务切片、终局错误、轨迹里最早的因果偏差、可能对应的机制，以及下一批数据准备干预什么。相同的 timeout 可能来自错误规划、工具调用失败或上下文管理失效；只按终局标签聚类，会把不同问题混在一起。

对这类评估，我会看五层证据：

1. **Outcome**：任务最终是否完成，能否由执行器或明确 verifier 判断。
2. **Process**：失败最早从哪里开始，后面的错误是否只是连锁反应。
3. **Slice**：问题是否在一类新任务上重复出现；单个 bad case 提供的证据还不够。
4. **Prescription**：它能否导出下一批任务、反馈形式或 Harness 变化。
5. **Transfer**：干预以后，提升是否出现在未参与数据生产的 private validation 上。

2. Ladder：用小实验决定要不要爬下一阶

假设一个 coding Harness 一周能跑出一百万条轨迹，要不要全做？这个问题不能拍脑袋决定。合成数据最麻烦的地方就在这里：样本可以写得很漂亮，答案也可以是对的，训练到目标模型上却没有任何变化。

Ladder 是一组成本逐级增加的受控实验。每一级只放大一个对象，其他条件尽量固定；相对 control 的信号足够稳定，才继续爬下一阶。它用小实验控制大规模投入的节奏，后续也可能从中总结出 scaling law。

这个词常用于 architecture scaling。更严谨的做法是在一组递增的 compute budget 上，分别训练 candidate 与 matched baseline；数据分布、objective、评估协议，以及模型规模与 token 的分配规则都预先固定。Architecture ladder 问的是：相对 **baseline** 的优势能否跨规模保持，是否值得进入下一档 **compute**？

Data ladder 换了放大对象。固定目标模型、训练配置与 eval，只增加某个固定来源的数据量或覆盖，观察每一批新增数据还能带来多少 held-out gain。它问的是：这批数据还有多少新的学习信号，边际回报在哪里开始消失？

Synthetic-data ladder 更麻烦，因为数据由 production recipe 现场生产。Producer、Harness、verifier、采样策略和 filter 共同决定数据分布；目标模型又决定这些轨迹能否被吸收。因此，一条可比较的 synthetic-data ladder 必须固定 **production recipe** 与 **target model**，只逐阶增加通过验证且去重后的轨迹量。Production recipe 的任一环节（producer、Harness、verifier、sampling policy 或 filter）或 target model 发生变化后，都应回到低成本台阶重新校准。

扩量本身还可能改变产线的输出：通过率下降、重复增加、简单样本淹没长尾。同一批轨迹放到两个目标模型上，也可能一个学到规划习惯，另一个只学到措辞。SynthLLM 的实验同样观察到 synthetic data 的饱和区会随目标模型规模变化（Qin et al., 2025）。所以“能生成多少”和“针对这个模型应该生成多少”是两个问题。

三种 Ladder 都是在做“小实验先行”，但它们的实验单位和曲线失效条件并不一样：

Ladder：用一串成本递增的受控实验，决定要不要爬下一阶段
每一阶只放大一个对象；控制条件一旦改变，就开启一条新的 ladder。

01

Architecture ladder

放大 training compute（模型与 token 按同一规则分配）

固定 candidate/baseline 对照、数据分布、objective、eval



看什么

相对 baseline 的优势能否跨 scale 保持，并在多个规模点复现。

02

Data ladder

放大 固定来源的数据量或覆盖

固定 target model、训练配置、eval



看什么

新增数据是否仍带来 held-out 增益，以及边际回报何时消失。

03

Synthetic-data ladder

放大 已验证、去重的轨迹量

固定 producer、Harness、verifier、filter、target model



为什么不同

数据是产线的输出，扩量时质量与分布也会漂移；换产线或目标模型，曲线即失效。

图 2. 每条 Synthetic-data ladder 都对应一条 production recipe 和一个 target model。

因此，synthetic-data ladder 是一串越来越贵的小赌注：先 audit 一小批任务与 verifier，再从同一 base checkpoint 做 pilot training；有可重复的 held-out signal 才扩量；增益接近噪声、成本或 regression risk 的阈值时先重复，跌破以后就停，或者修改产线并重开 ladder。

一个最小可用的实验设计

为了让 Ladder 可比较，每次只放开少数变量。最小版本固定 base checkpoint、优化器、训练轮数（或预先约定的数据曝光规则）和 held-out eval，只改变被接受且去重后的 trajectory 数量；总训练 token 随数据量增加。固定 training-token budget 时，实验测量的是等算力下的 coverage 或 mixture。纯数据量 scaling 会让总训练 token 随数据量增长，两种实验应分开报告。每一级至少重复两次，同时估计训练噪声。

每个点同时记录三类量：生产侧的生成成本、通过率和去重率；训练侧的有效 token、训练稳定性与行为变化；评估侧的目标切片提升、跨切片迁移和回归。只有当相邻台阶的增益在重复实验中方向一致，才进入更贵的一层。

在足够多的规模点、目标模型和生产管线上验证以前，称它为 **ladder experiment** 更准确，现有曲线也无法直接外推。Ladder 先用来控制投资节奏、暴露饱和点；稳定的 scaling law 需要更多实验支持。

每爬一阶，都重新问：新增这批轨迹还带来多少新能力？

阈值由重复实验的不确定性、production cost 与 regression risk 共同决定，各项目单独设定。

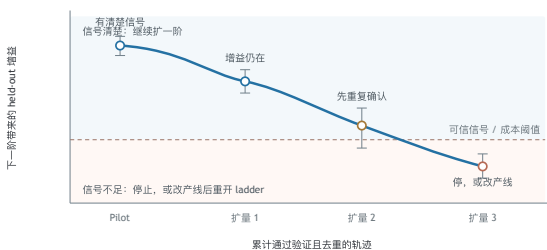


图 3. 决策规则示意。每个点都应从同一 base checkpoint 训练，并在未参与数据生产的 held-out 任务上重复测量；新的 production recipe 或 target model 需要重开一条曲线。

先看哪几个数

先看三组数：多少条通过验证，去重以后还剩多少，每增加一批不同的轨迹，独立 eval 上有没有新的提升。前两个数决定这条产线到底多贵，最后一个数决定它值不值得继续。总生成量只提供背景。

小批量训练看到信号以后，可以把被接受且不重复的轨迹逐级放大。每一级尽量固定模型起点、训练配置和评估集，也保留上一层作对照。如果量翻了几倍，独立任务却不再变好，或者通过率一路掉，那就已经接近这条 pipeline 的饱和点。

合理产量落在饱和点之前。没有 Ladder，很容易先把昂贵的数据做完，再回来解释它为什么没用。

Ladder 属于产线与目标模型

同一批 synthetic data 放到两个模型上，结果可能差很多。因为每条数据都带着生产模型和 Harness 的习惯：怎么拆题、爱用什么工具、什么时候回退、会犯什么错。目标模型能不能吸收这些习惯，并没有通用答案。

所以无论换目标模型，还是改 producer、Harness、verifier、sampling policy 或 filter，都应该先回到低成本台阶重跑。上一条曲线可以当参考，不能当保证。这也是 synthetic data 的 scaling law 比自然数据更难做的地方。

3. 轨迹需要一条靠谱的产线

以前的数据其实一直是人设计的。人决定去哪里找，什么值得留下，怎么标注，哪些任务先学、哪些后学。即使原料来自互联网，进入模型以前也已经走过一条人为的数据管线。

当公开网页逐渐退出主要增量来源，数据生产会从“收集文档”转向“记录工作”。一个完整的专家流程可能包含私有工具、反复沟通、局部判断、失败恢复和最终验收；只保存输入与最后答案，会丢掉最有训练价值的部分。数据基础设施要同时承担存储、标注和过程记录，把环境、行动、反馈和结果一起留下来。

现在不少 synthetic data pipeline 仍是 prompt + model + filter。这能很快得到大量问答，但过程信息很薄。更有训练价值的部分包括：模型怎么搜、怎么试、看到报错以后怎么改、在哪一步发现自己走错了。

从 prompt 到产线

coding 里，人知道测试结果很重要；数学里，人会用证明检查、反例和难度递进；agent 任务里，人知道环境状态和工具返回不能丢。这些经验过去散在 prompt、脚本和研究者的脑子里。Harness 的作用，是把它们变成模型每次运行都会遇到的环境。

这里说的 Harness 包括 prompt、工具、上下文管理、工作流、持久状态、权限与验证逻辑。它决定模型如何观察、行动、保存结果和检查自己，也直接决定最终能收集到哪一种轨迹（Weng, 2026）。

能追踪 任务、生产模型、Harness、工具和 verifier 都有清楚的版本。	看得到过程 留下必要的状态、行动、工具反馈、失败和修改。
能验证 成功尽量来自执行结果或独立判断，也能解释为什么丢掉一条轨迹。	有差异 控制不同 Harness 和模型的比例，去掉重复轨迹与同一种表达。
能回测 每批数据都回到 Ladder 和独立 eval，测量目标模型学到了多少。	

Harness-Evolve 之后，再 SFT

Harness-Evolve 可以这样理解：先让模型在一个有工具、有反馈的环境里多试几次。它可以走不同路径，可以失败，也可以根据结果回来改。然后从这些尝试里，挑出正确、有代表性、彼此又不太一样的完整过程。

最后送进 SFT 的是这些更好的 trajectory。训练目标很具体：模型下一次遇到类似问题时，更容易走上靠谱的路径；该验证的时候验证，看到失败会修改，需要工具时知道怎么用。

STaR 提供了一个直接的先例：生成 rationale、保留能得到正确答案的路径，再把它们训练回模型（Zelikman et al., 2022）。Harness-Evolve 把候选空间从“文本推理过程”扩大到带状态的 agent trajectory，工具返回、文件变化、执行错误、回退和重新规划都成为数据的一部分。DGM 和 Self-Harness 主要优化 agent 或 harness；本文的产物是训练数据，目标模型在最后的 SFT 阶段更新（Zhang et al., 2025; Zhang et al., 2026）。

Harness 工具、上下文、反馈、记忆、验证	Evolve 多次尝试、变体、回退、修改、选择
Trajectory data 留下完整、可靠而且有差异的过程	SFT 把这些过程训练回目标模型

图 4. 四个方框是同一条数据产线的四个阶段。Evolve 改善候选轨迹，最后的 SFT 再把这些过程写回目标模型。

为什么需要很多种 Harness

一个 Harness 会长出一种固定的轨迹。强调测试驱动的 coding Harness，会留下“运行—报错—修改”的路径；强调批评与重写的 Harness，会留下更多自我检查；强调搜索的 Harness，则会留下分支比较。它们都可能有用，也都可能用久了变成套路。

更稳妥的做法是让几种 Harness 同时生产，再用 Ladder 测量目标模型对每一种轨迹及其混合配方的吸收效果。额外的实验成本可以降低单一 pattern 填满训练集的风险。

Harness 也有自己的 horizon

长线任务对 Harness 提出了新的要求。为今天的 agent 写的 harness code，未必能支持 RSI 所需的工作。很多系统默认一个任务能在单次运行或一个 context 里结束，工具调用是同步的，verifier 很快返回，成功也能用一个最终状态判断。这些假设对几十分钟的 coding task 可能够用，对持续几天的实验、跨多轮训练的数据工程，或者反馈要很久以后才出现的研究任务就不一定成立。

METR 用“人类专家完成同一任务所需的时间”来刻画 agent 的 task-completion horizon。这个指标代理任务难度，与 agent 实际运行的墙上时间分开计算。他们的结果说明，模型能否把很多局部能力可靠地串成更长的行动序列，本身就是一条重要的能力轴（METR, 2026）。如果任务 horizon 增长得比 harness horizon 更快，瓶颈可能转移到状态丢失、上下文膨胀、错误累积和无法恢复上。

RSI-oriented harness 至少还需要几种今天并不总是默认具备的能力：

- 持久状态：实验、代码、数据版本和未完成事项需要保存到 context 之外；中断以后要能从 checkpoint 恢复。
- 分层目标：长任务要拆成可验证的阶段，同时保留阶段之间的依赖，避免只优化眼前的小分数。
- 异步执行：训练、评测和数据生成可能跑数小时或数天，Harness 要能启动、监控、取消和重新接管后台任务。
- 延迟反馈：最终 reward 很晚才出现时，需要保存中间证据，并把失败追溯到最早产生偏差的步骤。
- 可演化但有边界：模型可以修改 workflow、context policy 和工具组合，但 verifier、权限与审计日志不应被同一个改进回路随意改写。

最近的长程 agent 工作也开始把 compact state、checkpoint、verifier-backed state transition 和 targeted recovery 放到中心，减少向 prompt 反复塞入完整交互历史（Wu et al., 2026）。沿着这个方向，Harness-Evolve 会同时推进两件事：在 Harness 里 evolve trajectory，并随着任务变长升级 承载轨迹的 **Harness**。否则产线只能稳定地产出当前 horizon 以内的数据。

数据合成还是需要先验

目标先验	知道“数学更强”大致由哪些能力组成，但不把它锁死在一套题上。
任务先验	知道什么任务值得出、难度怎么往上走、哪种失败最有信息。
验证先验	能执行就执行，能检查就检查，尽量别让生产数据的模型给自己打分。
探索先验	允许模型试错和回退，也要拦住没有意义的长轨迹与固定套路。
多样性先验	控制题型、工具、解法和生产模型的来源，不让一种 pattern 填满数据集。

这些先验指出哪里值得探索、什么反馈可信、哪些结果需要怀疑。模型最终还要学会迁移这些经验。面对一个新 topic 或新的模糊目标，它应该能参考已有方法，自行设计任务和环境，再找出下一批有用的数据。

这里还有一个很现实的风险：生产模型会复制自己。如果行动、判断、筛选都交给同一个模型，Evolve 很容易只是把它原来的 pattern 变得更浓。执行工具、独立 verifier、不同来源的生产模型、真实数据锚点和少量人工抽查，都是为了让这条产线别变成一个回音室。

另一个风险是 **selection-induced shortcut**。Verifier 只认最终答案时，筛选容易留下碰巧答对的短路径；只偏好长轨迹，又会奖励无效的展开。因此筛选最好同时约束 outcome、过程完整性、轨迹新颖度和成本，并保留被拒绝轨迹及原因。这些记录可以服务清洗，也能指导下一轮改 Harness 和出题。

它与现有工作的关系

可以把相邻工作按“被优化的对象”分开：

- **Reasoning self-training** 优化模型内化的推理路径。STaR 是最直接的例子。
- **Synthetic-data scaling** 研究某条生成方法随数据量和模型规模的收益与饱和，SynthLLM 属于这一类。
- **Dynamic evaluation** 持续搜索和刷新 benchmark；AutoBench 是其中一个例子。
- **Harness optimization** 修改 prompt、工具、上下文或工作流，让冻结模型在部署时做得更好。Self-Harness 把失败挖掘、修改与回归测试连成闭环。
- **Open-ended agent evolution** 保留多个候选 agent，在可执行反馈下继续分支和选择。DGM 与 AlphaEvolve 展示了 archive 和 evaluator 如何支撑这种搜索（Novikov et al., 2025）。
- 本文的 **Harness-Evolve** 位于这些方向的交叉处：借 harness 扩大轨迹搜索空间，借 verifier 和多样性规则选择轨迹，再把结果蒸馏回目标模型。它目前仍是一个待验证的研究假设。

下一步要验证三个因果问题：多 Harness 是否比单 Harness 产生更可迁移的数据；Ladder 能否提前预测大规模生产的收益；经过 SFT 的模型学到了多少可复用的过程，又复制了多少生成器和 verifier 的表面偏好。

这条 loop 怎么转起来

把前面三件事接起来以后，它其实是一条很普通的工作流：eval 找问题，Harness-Evolve 做轨迹，SFT 把轨迹训练回去，Ladder 判断这条数据方法还值不值得继续。

下一轮的数据由上一轮模型的失败决定。模型变强以后，旧题会失效，旧 Harness 也会饱和，于是系统必须继续换题、换环境、换数据。数据本身也进入了迭代。

01 先说想补什么 不必一开始就有精确分数，但要说清大致方向。	02 出一批新题 不断换题，找到当前模型会稳定卡住的地方。
03 设计 Harness 把人的领域经验写进工具、反馈、验证和探索规则。	04 跑出轨迹 让模型尝试、失败、修改，留下几种不同的完整过程。
05 筛选，再 SFT 只把可靠且有差异的轨迹训练回目标模型。	06 用 Ladder 验证 从小到大看吸收、迁移和饱和，再决定要不要继续。

判断这条 loop 是否值得做，可以先看一件事：模型参与发现失败、生产下一批训练经验以后，下一版有没有稳定变强。

目标、Harness、验证和停止条件，目前仍需要大量人的判断。先把其中一段做成可以测、可以扩、也可以推翻的实验，就能开始积累证据。完整自治可以留给后续。

仍未解决的问题

- 怎么知道一个模糊目标已经覆盖得够好？如何防止它悄悄换成另一套 Benchmark？
- 模型生成的 benchmark 和 validation set，怎样保持新颖又不把 generator 的偏好当成能力边界？
- 不同 Harness 的轨迹怎么去重、怎么配比，才不会互相抵消？
- 什么时候该继续扩量，什么时候其实应该先换生产模型或 Harness？
- 模型能不能逐渐参与 Harness 的设计，同时不让 eval 和数据一起塌掉？
- SFT 学到的到底是更好的解题过程，还是生产模型更隐蔽的表达习惯？

这些问题仍没有确定答案。下一步可以挑一个窄领域，把整条 loop 跑几轮，记录它在哪里失效。

References

1. Villalobos et al. (2024), *Will we run out of data? Limits of LLM scaling based on human-generated data.*
2. Singh et al. (2024), *Evaluation data contamination in LLMs: how do we measure it and (when) does it matter?*.
3. Zelikman et al. (2022), *STaR: Bootstrapping Reasoning With Reasoning.*
4. Qin et al. (2025), *Scaling Laws of Synthetic Data for Language Models.*
5. Zhang et al. (2025), *Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents.*
6. Novikov et al. (2025), *AlphaEvolve: A coding agent for scientific and algorithmic discovery.*
7. Zhang et al. (2026), *Self-Harness: Harnesses That Improve Themselves.*
8. Weng (2026), *Harness Engineering for Self-Improvement.*
9. DePue (2026), *A Stargate for Data.*
10. METR (2026), *Task-Completion Time Horizons of Frontier AI Models.*
11. Wu et al. (2026), *StructAgent: Harness Long-horizon Digital Agents with Unified Causal Structure.*
12. Akhtar et al. (2026), *When AI Benchmarks Plateau.*
13. Center for AI Safety & Scale AI (2025), *Humanity's Last Exam.*
14. Li et al. (2025), *AutoBench: Creating Salient, Novel, Difficult Datasets for Language Models.*